



# MONASH University

MONASH UNIVERSITY

Faculty of Information Technology

FIT5032 Internet Applications Development

Assessed Lab 10

## API Service Integration

Weather and Library JSON APIs

|                          |                                                                                               |
|--------------------------|-----------------------------------------------------------------------------------------------|
| <b>Student Name</b>      | Han Wang                                                                                      |
| <b>Student ID</b>        | 36668664                                                                                      |
| <b>Tutorial/Lab</b>      | Tuesday 2:00 pm                                                                               |
| <b>Project Name</b>      | hwang-library                                                                                 |
| <b>GitHub Repository</b> | <a href="https://github.com/kokyuu/hwang-library">https://github.com/kokyuu/hwang-library</a> |

30 July 2026

# 1. Overview

This report documents the completed FIT5032 Assessed Lab 10 implementation in the existing hwang-library Vue 3 project. The application integrates an external weather service, supports browser-location and city-based weather requests, calculates author and book statistics from the Lab 2 dataset, and exposes all books as a JSON-formatted API page.

| Assessment item         | Implemented evidence                              |
|-------------------------|---------------------------------------------------|
| Task 10.1 - weather     | Current-location method and weather result UI     |
| Task 10.1 - statistics  | 3 authors and 6 books in CountBookAPI             |
| Task 10.2 - city search | Clayton, AU result in Celsius with a weather icon |
| Task 10.2 - all books   | 6 books shown as JSON on GetAllBookAPI            |
| Version control         | Dedicated lab10-api branch and focused commits    |

## 2. Page Setup and Routing

The application uses Vue Router and a shared BHeader navigation component. The route name GetWeather maps to /WeatherCheck, while the two library response pages are available at /CountBookAPI and /GetAllBookAPI.

### Lab 10 routes

src/router/index.ts

```

1  import { createRouter, createWebHistory } from 'vue-router'
2  import CountBookAPI from '../views/CountBookAPI.vue'
3  import GetAllBookAPI from '../views/GetAllBookAPI.vue'
4  import WeatherView from '../views/WeatherView.vue'
5
6  const router = createRouter({
7    history: createWebHistory(import.meta.env.BASE_URL),
8    routes: [
9      { path: '/', redirect: '/WeatherCheck' },
10     { path: '/WeatherCheck', name: 'GetWeather', component: WeatherView },
11     { path: '/CountBookAPI', name: 'CountBookAPI', component: CountBookAPI },
12     { path: '/GetAllBookAPI', name: 'GetAllBookAPI', component: GetAllBookAPI },
13   ],
14 })
15
16 export default router

```

Figure 1. Vue Router configuration for all Lab 10 pages.

## 3. Task 10.1 - Weather API

### 3.1 Current location and city controls

The WeatherView component validates a city search, handles loading and error states, and uses the browser Geolocation API for the current-location workflow. Coordinates are sent only after the user selects the Use current location control.

#### City search and browser geolocation

src/Views/WeatherView.vue

```

38  async function searchByCity() {
39    const query = city.value.trim()
40    if (!query) {
41      error.value = 'Enter a city name to search.'
42      return
43    }
44
45    await loadWeather(() => searchWeatherByCity(query))
46  }
47
48  async function fetchCurrentLocationWeather() {
49    if (!navigator.geolocation) {
50      error.value = 'Geolocation is not supported by this browser.'
51      return
52    }
53
54    loading.value = true
55    error.value = ''
56    navigator.geolocation.getCurrentPosition(
57      async ({ coords }) => {
58        await loadWeather(() => getWeatherByCoordinates(coords.latitude, coords.longitude))
59      },
60      (geolocationError) => {
61        loading.value = false
62        error.value = `Unable to use current location: ${geolocationError.message}`
63      },
64      { enableHighAccuracy: false, timeout: 10000 },
65    )
66  }
67
68  async function loadWeather(loader: () => Promise<WeatherResult>) {
69    loading.value = true
70    error.value = ''
71    try {
72      weatherData.value = await loader()
73    } catch (requestError) {
74      const message = requestError instanceof Error ? requestError.message : 'Weather request failed.'
75      error.value = message
76    } finally {
77      loading.value = false
78    }
79  }

```

Figure 2. City-search and browser-geolocation methods in WeatherView.vue.

### 3.2 Search weather by city

The external service accepts the assessed-lab example Clayton, AU. Country-code matching prevents the request from selecting a same-named city in another country. The returned Celsius temperature and weather code drive the corresponding weather icon.

## Weather API request and country matching

src/services/weather.ts

```

92 export async function searchWeatherByCity(query: string): Promise<WeatherResult> {
93   if (openWeatherKey) return fetchOpenWeather({ q: query })
94
95   const [cityName, requestedCountry] = query.split(',').map((part) => part.trim())
96   const response = await axios.get('https://geocoding-api.open-meteo.com/v1/search', {
97     params: { name: requestedCountry ? query : cityName, count: 10, language: 'en', format: 'json' },
98   })
99   const results = (response.data.results ?? []) as OpenMeteoPlace[]
100   const countryMatch = requestedCountry
101     ? results.find(
102       (place) =>
103         place.country_code?.toLowerCase() === requestedCountry.toLowerCase() ||
104         place.country?.toLowerCase() === requestedCountry.toLowerCase(),
105     )
106     : undefined
107   const place = countryMatch ?? results[0]
108   if (!place) throw new Error('No weather location found for "${query}"')
109
110   return fetchOpenMeteoWeather(
111     place.latitude,
112     place.longitude,
113     place.name,
114     place.country_code ?? place.country,
115   )
116 }
117

```

Figure 3. Weather API request, country matching and result mapping.

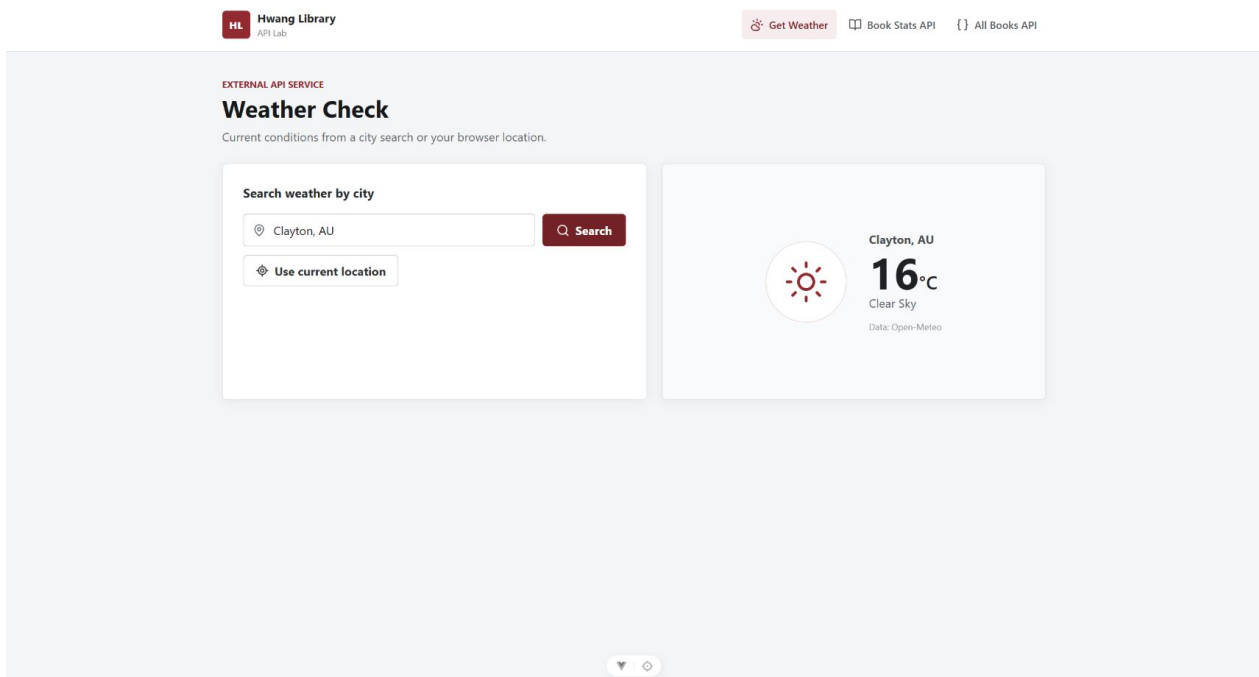


Figure 4. Browser result for Clayton, AU in Celsius with a clear-sky icon.

## 4. Task 10.1 - Author and Book Statistics

CountBookAPI imports the Lab 2 authors.json file, calculates the number of authors, reduces the nested famousWorks arrays into a total book count, and returns per-author counts. The response correctly reports 3 authors and 6 books.

```

Author and book statistics response
src/views/CountBookAPI.vue

1  <script setup lang="ts">
2  import { computed } from 'vue'
3  import authors from '../assets/json/authors.json'
4
5  const apiResponse = computed(() => ({
6    success: true,
7    data: {
8      authorsCount: authors.length,
9      totalBooks: authors.reduce((total, author) => total + author.famousWorks.length, 0),
10     authors: authors.map((author) => ({
11       name: author.name,
12       bookCount: author.famousWorks.length,
13     })),
14   },
15   }))

```

Figure 5. CountBookAPI response calculation from authors.json.

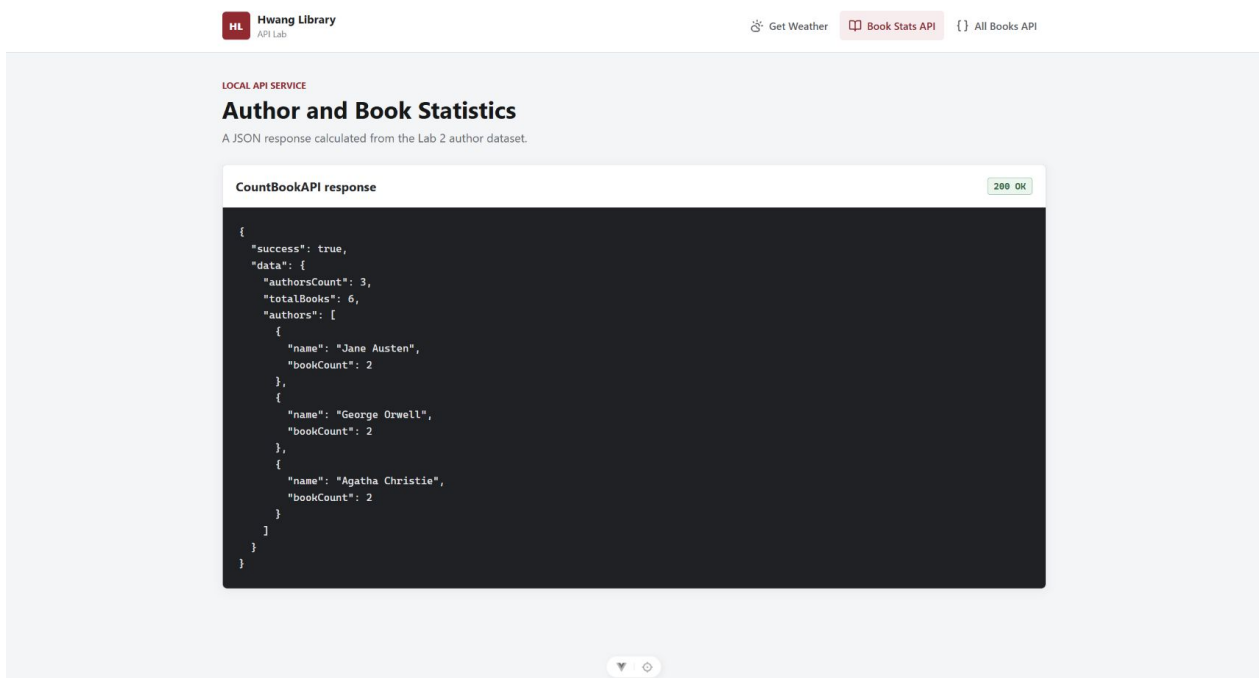


Figure 6. Browser response showing authorsCount 3 and totalBooks 6.

## 5. Task 10.2 - GetAllBookAPI

GetAllBookAPI flattens every author's famousWorks array and preserves the title, publication year, author name and author ID. The page presents the complete six-book collection as a JSON response at the required GetAllBookAPI route.

```

GetAllBookAPI response
src/views/GetAllBookAPI.vue

1  <script setup lang="ts">
2  import { computed } from 'vue'
3  import authors from '../assets/json/authors.json'
4
5  const apiResponse = computed(() => ({
6    success: true,
7    count: authors.reduce((total, author) => total + author.famousWorks.length, 0),
8    books: authors.flatMap((author) =>
9      author.famousWorks.map((book) => ({
10        title: book.title,
11        year: book.year,
12        author: author.name,
13        authorId: author.id,
14      })),
15    ),
16  }))

```

Figure 7. GetAllBookAPI transformation and JSON response structure.

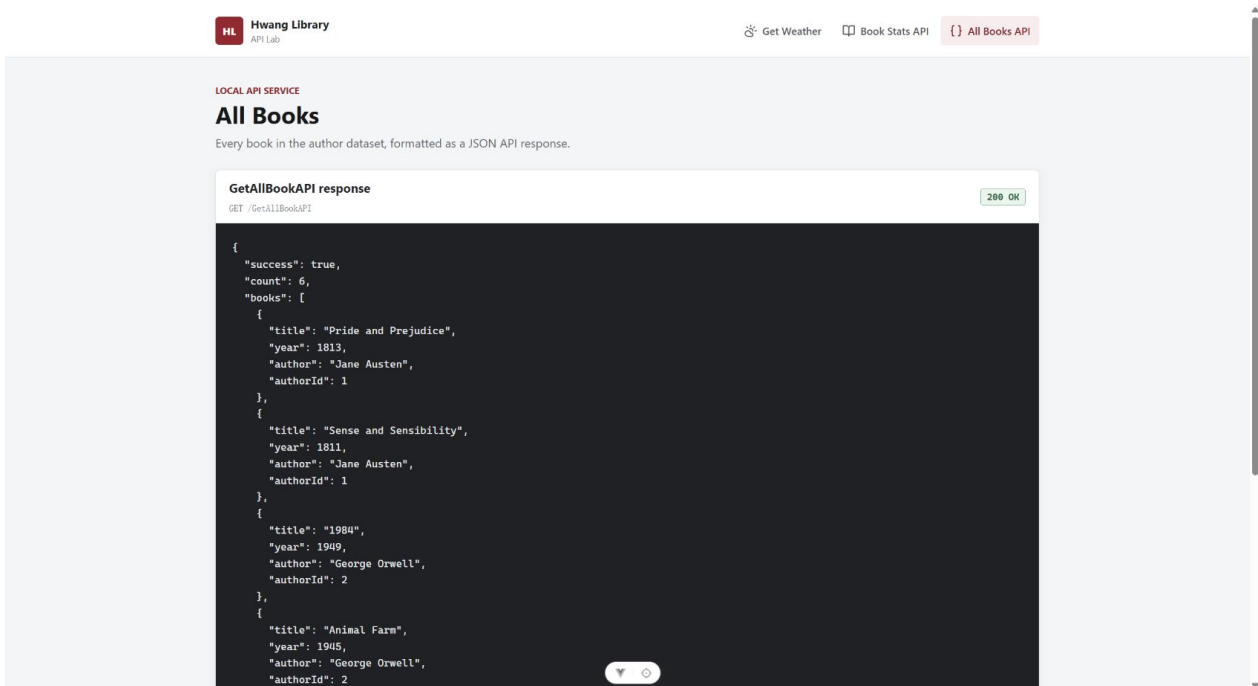


Figure 8. GetAllBookAPI browser response showing count 6 and the first response records.

```
{
  "title": "Pride and Prejudice",
  "year": 1813,
  "author": "Jane Austen",
  "authorId": 1
},
{
  "title": "Sense and Sensibility",
  "year": 1811,
  "author": "Jane Austen",
  "authorId": 1
},
{
  "title": "1984",
  "year": 1949,
  "author": "George Orwell",
  "authorId": 2
},
{
  "title": "Animal Farm",
  "year": 1945,
  "author": "George Orwell",
  "authorId": 2
},
{
  "title": "Murder on the Orient Express",
  "year": 1934,
  "author": "Agatha Christie",
  "authorId": 3
},
{
  "title": "Death on the Nile",
  "year": 1937,
  "author": "Agatha Christie",
  "authorId": 3
}
}
```

Figure 9. Continued browser response showing all remaining books and the closing JSON structure.

## 6. Verification and Responsive Layout

The project passes Vue TypeScript checking, the production Vite build, and the Vitest suite. The weather-code unit test verifies clear, rain, snow and thunderstorm mappings. Browser checks also confirmed that all navigation labels, controls and results remain readable at a 390-pixel mobile viewport.

```

Weather-code unit test
src/ __tests__ /weather.spec.ts

1  import { describe, expect, it } from 'vitest'
2  import { describeWeatherCode } from '../services/weather'
3
4  describe('describeWeatherCode', () => {
5    it('maps clear, rain, snow and storm weather codes', () => {
6      expect(describeWeatherCode(0)).toEqual({ description: 'Clear sky', kind: 'clear' })
7      expect(describeWeatherCode(61).kind).toBe('rain')
8      expect(describeWeatherCode(75).kind).toBe('snow')
9      expect(describeWeatherCode(95).kind).toBe('storm')
10    })
11  })

```

Figure 10. Unit coverage for external weather-code mapping.

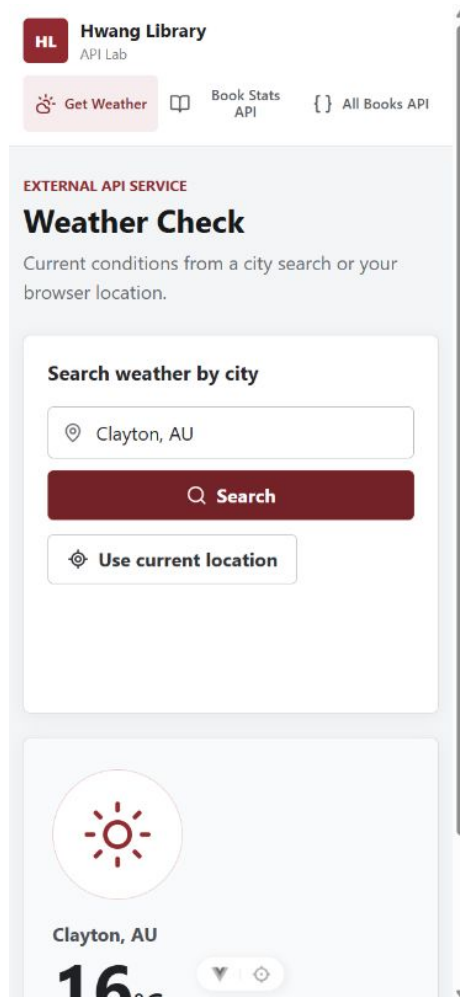


Figure 11. Responsive Weather Check page at a 390-pixel mobile viewport.

## 7. Git Version Control

Lab 10 was developed on the dedicated lab10-api branch so the existing Lab 2 and Lab 3 history remained intact. The implementation and automated verification were recorded in separate focused commits before the report was added.

```

Git commit history
branch: lab10-api

* bf24114 (HEAD -> lab10-api, origin/lab10-api) Add Lab 10 tests and setup documentation
* e405141 Implement Lab 10 weather and book APIs
* 1f08632 (origin/main, origin/HEAD, main) Delete
* 71bf5ae Complete Lab 3 Bootstrap user information form
* b81093e Set up Bootstrap 5
* 4c9aca6 Create basic user information form
* 56e86c4 Finalize commit
* f6ae68a Add George Orwell highlight binding
* 5e433b2 Add conditional rendering activities
* 2a943dd Add v-for rendering activities
* 7c6e991 Add computed properties for author data
* 75c53af Set up Vue project and import JSON files
  
```

Figure 12. Git history showing the focused Lab 10 implementation and test commits.

The screenshot displays the GitHub commit history for the lab10-api branch. The interface includes a search bar at the top and a navigation menu. The commits are listed in a table-like format, grouped by date. The most recent commits are from July 30, 2026, including 'Add Lab 10 tests and setup documentation' (bf24114) and 'Implement Lab 10 weather and book APIs' (e405141). Other commits include a 'Delete' commit (1f08632) from July 11, 2026, and several commits from July 8, 2026, related to Lab 3 Bootstrap user information form and Bootstrap 5 setup. Commits from July 1, 2026, include 'Finalize commit' (56e86c4). Commits from June 30, 2026, include 'Add George Orwell highlight binding' (f6ae68a), 'Add conditional rendering activities' (5e433b2), and 'Add v-for rendering activities' (2a943dd).

Figure 13. GitHub commit history for the pushed lab10-api branch.

## 8. Conclusion

The completed Lab 10 application satisfies the Pass/Credit and Distinction/High Distinction functional criteria. It can retrieve weather using browser coordinates or a city query, displays temperatures in Celsius with a matching icon, calculates the author and book totals from the existing dataset, and shows all books as valid JSON. The implementation is tested, responsive, documented and maintained through a clear Git commit history.